

EDS Lab Assignments

Practical2

Name: Harshal Dhanait

PRN:202501040049

2.1.1 List Operations

Problem Statement:

Write a Python program that implements a menu-driven interface for managing a list of integers. The program should have the following menu options:

1. Add
2. Remove
3. Display
4. Quit

The program should repeatedly prompt the user to enter a choice from the menu. Depending on the choice selected, the program should perform the following actions:

- **Add:** Prompts the user to enter an integer and add it to the integer list. If the input is not a valid integer, display "Invalid input".
- **Remove:** Prompts the user to enter an integer to remove from the list. If the integer is found in the list, remove it; otherwise, display "Element not found". If the list is empty, display "List is empty".
- **Display:** Displays the current list of integers. If the list is empty, display "List is empty".
- **Quit:** Exits the program.
- The program should handle invalid menu choices by displaying "Invalid choice". Ensure that the program continues to prompt the user until they choose to quit (option 4).

Code:

```
l1 = []
while True:
    print("1. Add")
    print("2. Remove")
    print("3. Display")
    print("4. Quit")

    n=int(input("Enter choice: "))
    if n==1:
        add = int(input("Integer: "))
        l1.append(add)
```

```

        print(f"List after adding: {l1}")
    elif n==2:
        if len(l1)==0:
            print("List is empty")
        elif len(l1)!=0:
            remove=int(input("Integer: "))
            if remove not in l1:
                print("Element not found")
            else:
                l1.remove(remove)
            print(f"List after removing: {l1}")
    elif n==3:
        if len(l1)==0:
            print("List is empty")
        else:
            print(l1)
    elif n==4:
        break
    else:
        print("Invalid choice")

```

Output:

The screenshot displays the CodeTANTRA IDE interface. On the left, the problem statement for '2.1.1. List operations' is shown, detailing the requirements for a menu-driven program to manage a list of integers. The main editor shows the Python code implementing these requirements. The right sidebar displays the test results, indicating that 2 out of 2 shown test cases and 1 out of 1 hidden test case(s) passed. Below the test results, the 'Test case 1' details are visible, showing the expected output and the actual output, which match perfectly.

Problem Statement: Write a Python program that implements a menu-driven interface for managing a list of integers. The program should have the following menu options: 1. Add, 2. Remove, 3. Display, 4. Quit. The program should repeatedly prompt the user to enter a choice from the menu. Depending on the choice selected, the program should perform the following actions:

- Add:** Prompts the user to enter an integer and add it to the integer list. If the input is not a valid integer, display "Invalid input".
- Remove:** Prompts the user to enter an integer to remove from the list. If the integer is found in the list, remove it; otherwise, display "Element not found". If the list is empty, display "List is empty".
- Display:** Displays the current list of integers. If the list is empty, display "List is empty".
- Quit:** Exits the program.
- The program should handle invalid menu choices by displaying "Invalid choice". Ensure that the program continues to prompt the user until they choose to quit (option 4).

Code Snippet:

```

1 # write the code..
2 l1=[]
3 while True:
4     print("1.Add")
5     print("2.Remove")
6     print("3.Display")
7     print("4.Quit")
8     n=int(input("Enter choice: "))
9     if n==1:
10        add = int(input("Integer: "))
11        l1.append(add)
12        print(f"List after adding:{l1}")
13    elif n==2:
14        if len(l1)==0:
15            print("List is empty")
16        elif len(l1)!=0:
17            remove=int(input("Integer: "))
18            if remove not in l1:

```

Test Results:

Metric	Value
Average time	0.064 s
Maximum time	0.102 s
Test Case 1 Status	Passed
Expected output	1. Add 2. Remove 3. Display 4. Quit Enter choice: 1 Integer: 11
Actual output	1. Add 2. Remove 3. Display 4. Quit Enter choice: 1 Integer: 11

2.1.2 Dictionary Operations

Problem Statement:

Write a Python program to perform insertion, update, deletion, and traversal operations on a dictionary. An initial dictionary containing 10 predefined records is already given in the program.

Operations to be Performed:

- 1.Insertion – Insert a new key-value pair into the dictionary using user input.
- 2.Update – Update the value of an existing key using user input.
- 3.Deletion – Delete a specified key from the dictionary using user input.
4. Traversal – Traverse the final dictionary and display all key-value pairs.

Input and Output Format:

- 1.Read an integer representing the key to be inserted and a string representing its value. Insert this new key-value pair into the dictionary and display the dictionary after insertion.
- 2.Read an integer representing the key to be updated and a string representing the new value. Update the value of the specified key (only if it exists) and display the dictionary after the update.
- 3.Read an integer representing the key to be deleted. Delete the specified key from the dictionary (only if it exists) and display the dictionary after deletion.
4. Finally, traverse the dictionary and display all key-value pairs.

The program should also display the original dictionary before performing any operations. Each output should be printed with appropriate messages.

Code:

```
# Initial dictionary with 10 predefined records
```

```
student = {  
    1: "Amit",  
    2: "Riya",  
    3: "Kiran",  
    4: "Neha",  
    5: "Arjun",  
    6: "Pooja",  
    7: "Rahul",  
    8: "Sneha",  
    9: "Vikram",  
    10: "Anjali"  
}
```

```
# write your code here..  
print("Original Dictionary:", student)
```

```
new_key = int(input())
```

```

new_value      =      input()
student[new_key] = new_value
print("After Insertion:", student)

update_key     =      int(input())
new_val = input()
if update_key in student:
    student[update_key] = new_val
print("After Update:", student)

del_key = int(input())
if del_key in student:
    del student[del_key]
print("After Deletion:", student)

print("Traversing Dictionary:")
for key, value in student.items():
    print(f"{key} : {value}")

```

Output:

The screenshot displays a coding platform interface with a dark theme. On the left, a sidebar shows the problem title "2.1.2. Dictionary Operations" and a brief description: "Write a Python program to perform insertion, update, deletion, and traversal operations on a dictionary. An initial dictionary containing 10 predefined records is already given in the program." Below this, the "Operations to be Performed:" section lists four tasks: Insertion, Update, Deletion, and Traversal. The "Input and Output Format:" section provides detailed instructions for each operation. A "Note:" section at the bottom left contains three bullet points. A "Sample Test Cases" button is located below the note.

The main area on the right shows the execution results. At the top, it indicates "2 out of 2 shown test case(s) passed" and "2 out of 2 hidden test case(s) passed". Below this, the "Test case 1" section displays the "Expected output" and "Actual output" side-by-side. The "Expected output" shows the original dictionary, followed by the results of inserting Suresh, updating Karthik's value, deleting Pooja, and traversing the dictionary. The "Actual output" shows the same sequence of operations performed on the initial dictionary. The results match the expected output, confirming the program's correctness.

2.2.1 Linear Search Technique

Problem Statement:

Write a program to check whether the given element is present or not in the array of elements using linear search.

Input format:

- The first line of input contains the array of integers which are separated by space
- The last line of input contains the key element to be searched

Output format:

- If the element is found, print the index.
- If the element is not found, print **Not found**.

Code:

```
def linear_search(arr, key):
    for i in range(len(arr)):
        if arr[i] == key:
            return i
    return -1

arr = list(map(int,input().split()))
key = int(input())

result = linear_search(arr, key)

if result != -1:
    print(result)
else:
    print("Not found")
```

Output:

The screenshot displays the CodeTANTRA IDE interface. On the left, the problem statement for '2.2.1. Linear search Technique' is visible, including input/output formats and a sample test case. The main editor shows the Python code for the linear search algorithm. The bottom right panel shows the execution results, indicating that 2 out of 2 shown test cases passed. The first test case shows an input array [1 2 3 4 3 5 6] and a key of 3, with the output being 2. The second test case shows an input array [1 2 3 4 3 5 6] and a key of 2, with the output being 1.

CodeTANTRA Home Learn Anywhere 202501040049@mitaoe.ac.in Support Logout

2.2.1. Linear search Technique

Write a program to check whether the given element is present or not in the array of elements using linear search.

Input format:

- The first line of input contains the array of integers which are separated by space
- The last line of input contains the key element to be searched

Output format:

- If the element is found, print the index. If the element found multiple times, print the starting index
- If the element is not found, print Not found.

Sample Test Case:

Input:

```
1 2 3 4 3 5 6
3
```

Output:

```
2
```

```
def linear_search(arr, key):
    for i in range(len(arr)):
        if arr[i] == key:
            return i
    return -1

arr = list(map(int,input().split()))
key = int(input())

result = linear_search(arr, key)

if result != -1:
    print(result)
else:
    print("Not found")
```

Average time: 0.006 s Maximum time: 0.006 s 2 out of 2 shown test case(s) passed

5.25 ms 6.00 ms 2 out of 2 hidden test case(s) passed

Test case 1 Expected output: 2 Actual output: 2

Test case 2 Expected output: 1 Actual output: 1

Terminal Test cases

PREV RESET SUBMIT NEXT

2.2.2 Captain of the Team

Problem Statement:

You are provided with the heights of 11 cricket players (in centimeters). Your task is to identify the tallest player, who will be selected as the captain of the team.

Input Format:

The first line of input will contain 11 integers, each representing the height of a player (in centimeters), each separated by a space.

Output Format

The output should be the height (in centimeters) of the tallest player.

Code:

```
heights = list(map(int,input().split()))
tallest_player = max(heights)
print(tallest_player)
```

Output:

The screenshot displays the CODETANTRA online coding interface. On the left, the problem statement for '2.2.2. Captain of the Team' is visible, including the input and output formats. The main editor shows the following Python code:

```
1 heights = list(map(int,input().split()))
2 tallest_player = max(heights)
3 print(tallest_player)
```

Below the code editor, the execution results are shown:

- Average time: 0.004 s, Maximum time: 0.007 s
- 4.33 ms, 7.00 ms
- 1 out of 1 shown test case(s) passed
- 2 out of 2 hidden test case(s) passed

Test Case 1 is expanded, showing the expected output and actual output:

Expected output	Actual output
171 169 185 156 174 191 186 190 187 172 160	171 169 185 156 174 191 186 190 187 172 160
191	191

At the bottom, there are buttons for 'PREV', 'RESET', 'SUBMIT', and 'NEXT'.